Introduction to Artificial Intelligence

Mihir Rao

Contents

1 Introduction to AI and Rational Agents **2**
1.1 Agents and their environments . . . 2
1.2 Reflex and planning agents . . . 2
1.3 Specifying a task with PEAS . . . 3
1.4 Properties of the environment . . . 3

2 Search: State Spaces and Uninformed Search **4**
2.1 Formulating a search problem . . . 4
2.2 State-space graphs and search trees . . . 5
2.3 A common tree-search procedure . . . 6
2.4 Depth-first search . . . 6
2.5 Breadth-first search . . . 7
2.6 Iterative deepening . . . 7
2.7 Uniform-cost search . . . 8

3 Search: Informed Search, A* and Heuristics **9**
3.1 Heuristics and remaining cost . . . 9
3.2 Greedy search and A* . . . 9
3.3 Admissibility and A* tree search . . . 10
3.4 Constructing heuristics by relaxation . . . 11
3.5 Dominance and combining estimates . . . 12
3.6 Graph search and repeated states . . . 13
3.7 Consistency and A* graph search . . . 14

4 Search: Local Search **15**
4.1 A* heuristics and memory . . . 15
4.2 Local search problems . . . 15
4.3 Hill climbing . . . 16
4.4 Simulated annealing . . . 17
4.5 Local beam search . . . 18
4.6 Genetic algorithms . . . 19
4.7 Continuous state spaces . . . 20

Lecture 1: Introduction to AI and Rational Agents

1.1 Agents and their environments

An AGENT observes an ENVIRONMENT through sensors and changes it through actuators. Its observations inform the actions it takes. The environment together with its agents forms the WORLD.

A RATIONAL AGENT chooses actions to obtain the best expected outcome according to its goals or preferences. The criterion concerns what can be expected from the available information, rather than a guarantee that every realized outcome will be favorable.

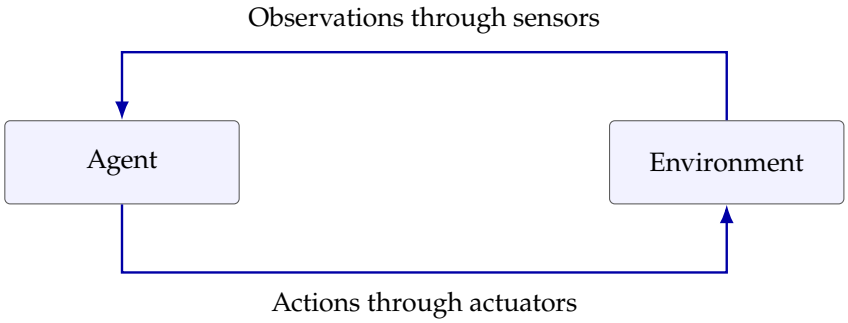


Figure 1: The interaction between an agent and its environment. Sensors supply observations; actuators carry out actions.

For a checkers-playing agent, the environment is the board and opposing player, and an action is a legal piece move. The agent's preferences determine which outcomes it should seek.

1.2 Reflex and planning agents

A REFLEX AGENT selects an action from its current percept, possibly using memory or an estimate of the current state. It does not compare the future consequences of candidate actions.

A PLANNING AGENT uses a model of how actions change the world to reason about possible futures. It predicts the consequences of actions and chooses among them according to its objective. The distinction is between reacting to the present and explicitly looking ahead; retaining memory alone does not make an agent a planner.

1.3 Specifying a task with PEAS

A PEAS DESCRIPTION fixes four components of the task:

Component	What it specifies
Performance measure	How outcomes are evaluated.
Environment	Where the agent acts and what affects it.
Actuators	How the agent changes the environment.
Sensors	How the agent obtains information.

The performance measure supplies the objective; the remaining components describe the circumstances and means of achieving it.

1.4 Properties of the environment

The information available to an agent and the effects of its actions determine what its design must handle.

- FULLY OBSERVABLE versus PARTIALLY OBSERVABLE: does the agent observe the relevant state completely? With partial observations, it must maintain an internal estimate of the state.
- DETERMINISTIC versus STOCHASTIC: does a state and action determine one next state, or a distribution of possible next states? Stochastic transitions require reasoning about uncertain outcomes.
- SINGLE-AGENT versus MULTI-AGENT: do other agents also act in the environment? Their behavior affects the outcome, and randomizing actions can prevent predictability.
- STATIC versus DYNAMIC: can the environment change while the agent is deciding what to do? In a dynamic environment, deliberation may leave observations outdated.
- KNOWN versus UNKNOWN dynamics: is the transition model available to the agent? Unknown dynamics require learning how actions affect the world, potentially through deliberate exploration.

Knowing the dynamics does not remove randomness: an agent can know a stochastic transition model while remaining uncertain about the next outcome.

Source. Nikhil Sharma, *CS 188: Introduction to Artificial Intelligence*, Spring 2024, Note 1: Agents.

Lecture 2: Search: State Spaces and Uninformed Search

Planning requires a model of how actions change the world and a test for whether a goal has been reached. A complete planning method finds a plan whenever one exists; an optimal method finds a plan of minimum cost. An agent can plan once or replan as new observations arrive. A reflex rule can also be rational when its chosen action already maximizes the relevant objective.

2.1 Formulating a search problem

Definition 2.1 [Search problem]. A search problem consists of a state space $\mathcal{S}$, available actions $A(s)$, a transition model $T(s, a)$, action costs $c(s, a)$, a start state s_0, and a goal test Goal(s). A plan is a sequence of actions that takes s_0 to a goal state. Its cost is the sum of its action costs.

The successor function combines the actions, their resulting states, and their costs. In route planning through Romania, states are cities and successors are adjacent cities, with road distances as costs. The start is Arad and the goal test asks whether the current city is Bucharest.

A world state describes the full environment. A search state retains the information needed for the chosen planning problem. This abstraction depends on the objective.

	Pathing	Eat-all-dots
State	Position (x, y)	Position and remaining-dot booleans
Actions	Legal north, south, east, west moves	The same legal moves
Transition	Update position	Update position and consumed dots
Goal test	Reach the destination	No dots remain

For 120 agent positions, four orientations, two ghosts with 12 positions each, and 30 independently represented dot booleans, the full encoding has

$$120 \cdot 4 \cdot 12^2 \cdot 2^{30}$$

possible configurations. The pathing abstraction has only 120 states; eat-all-dots has $120 \cdot 2^{30}$. These are counts of combinations in the encoding; restrictions on reachability can reduce the reachable subset.

To eat all dots while keeping ghosts continuously scared, the search state must also retain the remaining power pellets and the remaining scared time. Information can be omitted only when it is irrelevant to future decisions.

2.2 State-space graphs and search trees

A STATE-SPACE GRAPH has one vertex per state and directed edges for action results. A SEARCH TREE instead unfolds possible plans from the start state: its nodes represent paths, and their labels give the states reached by those paths. Different paths can therefore create different tree nodes with the same state label.

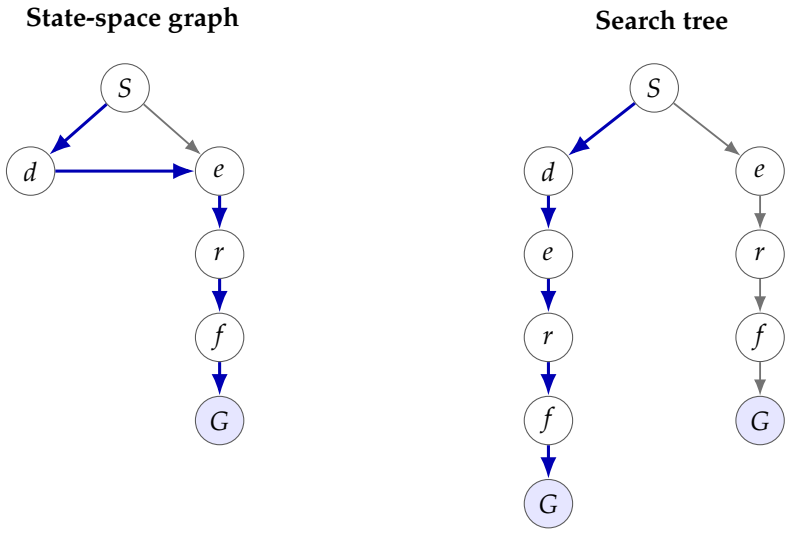


Figure 2: Selected transitions and their search tree. The state e occurs once in the graph but twice in the tree, reached by $S \to e$ and $S \to d \to e$. Blue marks $S \to d \to e \to r \to f \to G$.

Cycles in a finite graph can produce an infinite search tree. Neither structure needs to be constructed in full: generate successors only when a node is expanded, and retain the partial plans currently under consideration.

2.3 A common tree-search procedure

The FRONTIER, or FRINGE, contains generated nodes that have not yet been selected for exploration. A node records its state, parent, incoming action, and cumulative path cost g.

Algorithm 2.1: Tree search

Input: Search problem and an initially empty frontier F with a chosen removal rule.
Output: A plan reaching a goal, or failure if the frontier is exhausted.

Insert the root node $(s_0, \varnothing, \varnothing, 0)$ into F
while F is not empty **do**
    Remove a node n from F according to the removal rule
    if Goal(n.state) **then**
        return the plan obtained by following n's parent links
    for each $a \in A(n.\text{state})$ **do**
        $s' \leftarrow T(n.\text{state}, a)$
        $g' \leftarrow n.g + c(n.\text{state}, a)$
        Insert (s', n, a, g') into F
return failure

The removal rule determines the exploration strategy. This is tree search: it does not suppress repeated states. In particular, the goal test occurs when a node is removed, which matters for cost-sensitive search.

UNINFORMED SEARCH uses the problem's transitions and costs without an estimate of how close a state is to a goal. To compare methods, let $b > 1$ bound the branching factor, m be the maximum tree depth, and s be the shallowest goal depth. A full tree through depth m has

$$1 + b + b^2 + \cdots + b^m = O(b^m)$$

nodes. Time bounds below count tree-node operations, apart from the additional cost of implementing a priority queue.

2.4 Depth-first search

DEPTH-FIRST SEARCH (DFS) removes a deepest frontier node first, using a last-in, first-out STACK. Newly generated children are explored before older siblings.

With finite maximum depth m, DFS may explore the entire tree, taking $O(b^m)$ time. It retains the current branch and pending siblings along it, requiring $O(bm)$ space. It can fail to return when a branch is infinite, even if another branch contains a goal. Thus unrestricted tree-search DFS is not complete. It is also not optimal: the first goal in its exploration order need not be shallow or inexpensive.

2.5 Breadth-first search

Breadth-first search (BFS) removes a shallowest frontier node first, using a first-in, first-out queue. It finishes each depth before exploring the next one.

For finite b and a goal at finite depth s, BFS is complete and finds a shallowest goal. This is a least-cost plan when all action costs are equal and positive; unequal costs break the connection between depth and cost.

There are $1 + b + \cdots + b^s = O(b^s)$ nodes through the goal depth. With the goal test on removal, other nodes at depth s can generate children at depth $s + 1$ before a goal is selected. A bound on generated nodes and frontier storage is therefore $O(b^{s+1})$. The exponential frontier is BFS's main space limitation.

DFS can reach a deep goal quickly if its early branches are favorable; BFS is useful when a shallow solution exists or infinite branches would trap DFS. Their performance depends on the tree and successor order.

2.6 Iterative deepening

Iterative deepening search repeatedly runs depth-limited DFS, increasing the limit until a goal is found. A limit of zero checks only the start; subsequent runs allow depths $1, 2, \ldots$.

Although upper levels are revisited, the deepest level dominates for $b > 1$. To reach depth s, the work is bounded by

$$\sum_{i=0}^{s}(s - i + 1)b^i = b^s \sum_{j=0}^{s} \frac{j + 1}{b^j} = O(b^s).$$

Each run stores only its DFS frontier, giving $O(bs)$ space at the final limit. Finite branching makes it complete, and equal positive action costs make its first solution optimal.

For $b = 10$ and $s = 5$, counting full levels and omitting the root,

$$\begin{aligned} \text{BFS:} \quad & 10 + 100 + 1000 + 10000 + 100000 = 111110, \\ \text{iterative deepening:} \quad & 5 \cdot 10 + 4 \cdot 100 + 3 \cdot 1000 + 2 \cdot 10000 + 100000 = 123450. \end{aligned}$$

The repeated work is modest compared with the storage saved.

2.7 Uniform-cost search

Uniform-cost search (UCS) removes a node with minimum cumulative path cost $g(n)$, using a priority queue. It explores cost contours rather than depth levels. With equal positive action costs, this ordering agrees with BFS's depth ordering, up to ties.

Proposition 2.2 [UCS optimality]. *With nonnegative action costs, a goal returned by UCS has minimum plan cost.*

Proof. Suppose UCS removes a goal of cost C, but a cheaper goal path exists. There is still a frontier node on that path: initially its root is present, and expanding a prefix inserts its next prefix. Nonnegative costs make every prefix of the cheaper path cost at most its final cost, which is less than C. UCS would remove such a prefix before the goal of cost C, a contradiction. □

Optimality on return does not alone guarantee termination. If every action cost is at least $\epsilon > 0$, branching is finite, and an optimal plan has finite cost C^*, only finitely many tree nodes can precede that plan. UCS is then complete. A positive lower bound rules out endlessly exploring ever-longer paths whose costs stay below the goal cost.

Nodes expanded before termination have cost at most C^* and depth at most $d = \lfloor C^*/\epsilon \rfloor$. There are $O(b^d)$ such nodes; their generated children give time and space bounds $O(b^{d+1})$. Thus the effective depth is set by C^*/ϵ. Maintaining a heap adds a logarithmic factor per priority-queue operation.

UCS uses no information about the goal's location, so it can explore many cheap paths in unhelpful directions. DFS, BFS, and UCS share the same tree-search procedure; their frontier removal rules account for their different behavior.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Spring 2024, Lecture 2: Search; Nikhil Sharma, Note 2.

Lecture 3: Search: Informed Search, A* and Heuristics

Informed search uses an estimate of the remaining cost to guide which partial plan to explore. The search problem and successor function stay the same; the frontier ordering changes.

3.1 Heuristics and remaining cost

A heuristic $h(s)$ estimates the cheapest cost from state s to a goal. Let $g(n)$ be the actual cost of the partial plan represented by search node n. We abbreviate $h(n) = h(n.\text{state})$: the heuristic depends on the state, while g depends on the path used to reach it.

For unit-cost movement on a four-neighbor grid, two estimates are

$$h_{\mathrm{M}} = |x - x_G| + |y - y_G|, \qquad h_{\mathrm{E}} = \sqrt{(x - x_G)^2 + (y - y_G)^2}.$$

Manhattan distance ignores walls; Euclidean distance also permits a straight-line route. For horizontal and vertical displacements 10 and 5, these give 15 and $\sqrt{125} \approx 11.2$. In road routing, straight-line distance is similarly a lower bound on driving distance; Arad is 366 km from Bucharest by this estimate.

Example: *Pancake sorting.* A state records pancake sizes from top to bottom. An action reverses the top k pancakes and costs k, rather than one. The goal is increasing order. For example,

$$(4, 1, 2, 3) \xrightarrow{\text{flip 4}} (3, 2, 1, 4) \xrightarrow{\text{flip 3}} (1, 2, 3, 4)$$

has cost $4 + 3 = 7$. The largest pancake still out of place gives a heuristic: 4, then 3, then 0 along this plan. If pancake k is out of place, some future flip must reach its target position k, costing at least k. Thus this estimate never exceeds the remaining sorting cost.

3.2 Greedy search and A*

These methods use the same priority-queue search procedure with different priorities:

Method	Remove a node minimizing	Information used
Uniform-cost search	$g(n)$	Cost already incurred
Greedy search	$h(n)$	Estimated remaining cost
A* search	$f(n) = g(n) + h(n)$	Estimated total plan cost

Greedy search can reach a goal quickly, but ignores what it has already spent. It is not optimal, and on infinite search trees it need not be complete. With straight-line distance, the route Arad–Sibiu–Fagaras–Bucharest costs 450 km, while the route through Rimnicu Vilcea and Pitesti costs 418 km.

For the following illustration, break greedy-search ties in favor of the most recently inserted node.

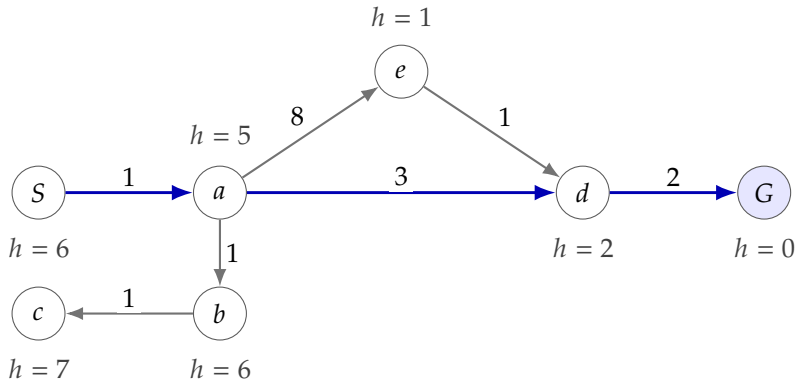


Figure 3: Edge labels are costs; labels beside nodes are heuristic values. The blue route costs 6. Greedy search instead takes S, a, e, d, G, costing 12.

In this graph, UCS explores b and c before reaching d. A* selects a, then d with $f(d) = 4 + 2 = 6$, then the goal of cost 6. The other frontier choices have $f(b) = 2 + 6 = 8$ and $f(e) = 9 + 1 = 10$. Greedy instead chooses e after a because $h(e) = 1$ is smallest.

Note. Test for a goal when removing a node, not when generating it. For paths $S \to A \to G$ with costs $(2, 2)$ and $S \to B \to G$ with costs $(2, 3)$, set $h(A) = 2$, $h(B) = 1$, and $h(G) = 0$. A* explores B first and generates a goal of cost 5, but A remains on the frontier with $f(A) = 4$. Continuing finds the cheaper goal of cost 4.

Arbitrary scores can discard cost information entirely: assigning $h(n) = 1 - g(n)$ makes every priority $f(n) = 1$. Behavior then depends on tie-breaking; FIFO ties produce BFS, which need not minimize unequal costs. This path-dependent score is not a valid nonnegative state heuristic in general.

3.3 Admissibility and A* tree search

Definition 3.1 [Admissible heuristic]. Let $h^*(s)$ be the minimum remaining cost from s to a goal. A heuristic is admissible if

$$0 \le h(s) \le h^*(s) \qquad \text{for every state } s.$$

In particular, $h(G) = 0$ at every goal.

Overestimation can hide a good plan. Suppose $S \to A$ costs 1, $A \to G$ costs 3, and $S \to G$ costs 5. With $h(S) = 7$, $h(A) = 6$, and $h(G) = 0$, A* returns the direct cost-5 goal before exploring A, whose priority is $1 + 6 = 7$. The actual optimum is 4.

Theorem 3.2 [Optimality of A* tree search]. *With an admissible heuristic and nonnegative action costs, a goal returned by A* tree search has minimum plan cost.*

Proof. Let C^* be the cost of an optimal plan. Until an optimal goal is removed, some prefix n of that plan remains on the frontier. Admissibility gives

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n) \leq C^*.$$

Any suboptimal goal B has $f(B) = g(B) > C^*$, since $h(B) = 0$. It cannot be removed while that cheaper prefix remains. Hence a returned goal cannot be suboptimal. □

For completeness on an infinite search space, also assume finite branching, a positive lower bound ϵ on action costs, and a finite-cost optimal plan. Every node selected before that goal has $g(n) \leq f(n) \leq C^*$, so its depth is at most $\lfloor C^*/\epsilon \rfloor$; there are only finitely many such nodes.

UCS explores cost contours in all directions. A* uses its lower bound to concentrate on plausible routes while retaining alternatives needed for optimality. Applications include routing, games, resource and robot-motion planning, translation, and speech recognition.

3.4 Constructing heuristics by relaxation

A relaxed problem removes constraints or permits additional actions without increasing the costs of existing actions. Every original plan is still available, so the exact relaxed optimum is an admissible heuristic for the original problem. Manhattan distance on a maze is the exact distance after removing its walls.

In the 8-puzzle, a state places eight labeled tiles and one blank on a 3×3 board. A unit-cost action slides a tile adjacent to the blank into it. The full encoding has 9! arrangements, and the center blank in the displayed start state permits four actions.

Start

7	2	4
5		6
8	3	1

Goal

	1	2
3	4	5
6	7	8

Figure 4: The 8-puzzle example. The blank is excluded from both heuristic sums: the start has eight misplaced tiles and total Manhattan distance 18.

Two admissible estimates are

$$h_{\text{tiles}}(s) = \#\{\text{numbered tiles outside their goal positions}\},$$
$$h_{\text{Manhattan}}(s) = \sum_{t=1}^{8} \left(|x_t - x_t^G| + |y_t - y_t^G| \right).$$

Each misplaced tile must move at least once. Each tile must also travel at least its Manhattan displacement, and one action moves only one tile by one square. These give the two lower bounds. Equivalently, allow each tile to move to its goal directly, or to move freely through other tiles while paying one per grid step.

The following illustrative averages from Andrew Moore count expanded nodes by optimal solution depth:

Search / heuristic	4 steps	8 steps	12 steps
UCS	112	6,300	3.6×10^6
A* / misplaced tiles	13	39	227
A* / Manhattan distance	12	25	73

The exact heuristic h^* is admissible too, but computing it may require solving the original problem. Search effort trades off against the work needed to evaluate the heuristic.

3.5 Dominance and combining estimates

For admissible heuristics, h_a DOMINATES h_b when $h_a(s) \geq h_b(s)$ for every state. Its lower bound is at least as informative. In the 8-puzzle,

$$0 \leq h_{\text{tiles}} \leq h_{\text{Manhattan}} \leq h^*.$$

The zero heuristic reduces A* to UCS. Larger lower bounds can reduce expansions, though evaluation cost and tie-breaking also affect performance.

Proposition 3.3 [MAXIMUM OF ADMISSIBLE HEURISTICS]. *For admissible $h_1, \ldots, h_k$, $h(s) = \max_i h_i(s)$ is admissible and dominates every h_i.*

Proof. Every $h_i(s)$ lies in $[0, h^*(s)]$, so their maximum does too. By definition, it is at least each individual value. □

Thus pointwise maximum is the least upper bound in the dominance order. The zero heuristic is the bottom element and h^* the top.

3.6 Graph search and repeated states

Different paths can reach the same state, creating repeated work or infinite loops in tree search. Graph search maintains a closed set of expanded states and skips later copies of them. Use a hash set for expected constant-time membership, rather than a list. Suppressing repeated states preserves reachability because a state's available continuations do not depend on which path reached it.

Algorithm 3.1: A* graph search without reopening closed states

Input: Search problem and heuristic h.
Output: A goal plan, or failure if the frontier is exhausted.

```
C ← ∅; insert the root node with g = 0 into frontier F
while F is not empty do
    Remove a node n minimizing n.g + h(n.state)
    if n.state ∉ C then
        if Goal(n.state) then
            return n's plan
        Add n.state to C
        for each a ∈ A(n.state) do
            s' ← T(n.state, a)
            g' ← n.g + c(n.state, a)
            Insert child (s', n, a, g') into F
return failure
```

The closed set records states on expansion, not on first generation. However, even this rule can lose optimality with an admissible heuristic.

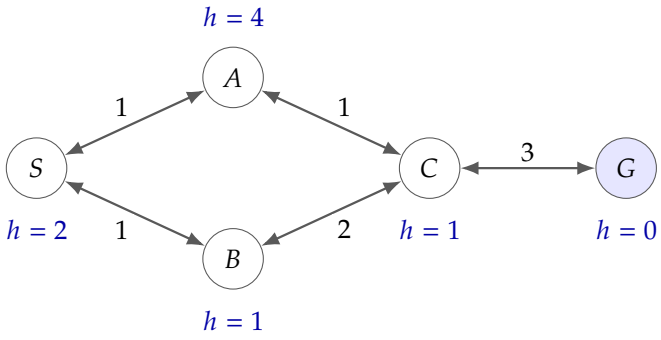


Figure 5: An admissible but inconsistent heuristic. Edge costs apply in both directions. Closing C through B prevents the later, cheaper arrival through A from being expanded.

The expansion order is S, B, C, A, G. The first arrival at C has $g(C) = 3$ and $f(C) = 4$, beating $f(A) = 1 + 4 = 5$. Expanding A later generates a cheaper arrival at C with $g = 2$, but C is already closed. The returned route costs $1 + 2 + 3 = 6$; the optimum $S \to A \to C \to G$ costs $1 + 1 + 3 = 5$.

3.7 Consistency and A* graph search

Definition 3.4 [Consistent heuristic]. A nonnegative heuristic, zero at every goal, is consistent if for every directed edge $u \to v$ of cost $c(u, v)$,

$$h(u) \le c(u, v) + h(v).$$

For a bidirectional edge, the condition must hold in both directions.

Consistency bounds the drop in heuristic value across each action. In the preceding example, $h(A) - h(C) = 4 - 1 = 3 > 1 = c(A, C)$. Replacing $h(A)$ by 2 makes the displayed heuristic consistent.

Consistency implies admissibility: summing its inequalities along any path to a goal G gives $h(s) \le \text{cost}(s \rightsquigarrow G) + h(G)$. Since $h(G) = 0$, taking a cheapest such path yields $h(s) \le h^*(s)$. It also makes f nondecreasing along a path: for a child n' of n,

$$f(n') = g(n) + c(n, n') + h(n') \ge g(n) + h(n) = f(n).$$

Theorem 3.5 [Optimality of A* graph search]. *With a consistent heuristic and nonnegative costs, a state is first expanded along a cheapest path to it. Consequently, a returned goal is optimal even when closed states are never reopened.*

Proof. Suppose v is the first state closed with a suboptimal path cost $g(v) > g^*(v)$, where g^* is shortest-path cost from the start. On a shortest path to v, let u be the first state not yet closed. Its predecessor was closed optimally and inserted a node for u with cost $g^*(u)$, so that node is still on the frontier. Consistency along the remaining path gives

$$g^*(u) + h(u) \le g^*(v) + h(v) < g(v) + h(v).$$

That node must be selected before the suboptimal node for v, a contradiction. A selected goal therefore has optimal path cost. □

Both 8-puzzle heuristics are consistent: one unit-cost move changes each estimate by at most one. Pointwise maximum also preserves consistency. If i maximizes the value at u, then

$$\max_j h_j(u) = h_i(u) \le c(u, v) + h_i(v) \le c(u, v) + \max_j h_j(v).$$

Thus combining consistent heuristics by maximum retains the graph-search guarantee. These guarantees concern the search model; even an optimal modeled plan can fail if its transitions or costs misdescribe the real world.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 3 (Spring 2024); Nikhil Sharma, Note 3.

Lecture 4: Search: Local Search

4.1 A* heuristics and memory

A useful heuristic often combines several simple lower bounds by the pointwise maximum, which stays admissible.

Example: *Knight's moves.* Count the fewest knight moves from S to G, whose displacement is $(\Delta x, \Delta y)$. One move changes the Manhattan distance to G by at most 3, the Euclidean distance by at most $\sqrt{5}$, and $\max(|\Delta x|, |\Delta y|)$ by at most 2. Hence

$$h_1 = \frac{|\Delta x| + |\Delta y|}{3}, \qquad h_2 = \frac{\sqrt{\Delta x^2 + \Delta y^2}}{\sqrt{5}}, \qquad h_3 = \frac{\max(|\Delta x|, |\Delta y|)}{2}$$

are lower bounds. Every move changes the square's color, so the move count is even exactly when S and G have the same color. Rounding each bound up to the next integer of this parity keeps it admissible, and so does the maximum $h = \max(h'_1, h'_2, h'_3)$ of the rounded bounds. Between opposite corners of a 5×10 board, $(\Delta x, \Delta y) = (4, 9)$ gives 4.33, 4.40, and 4.5; all round up to the odd value 5, which is the true minimum: $(1, 2), (1, 2), (1, 2), (-1, 2), (2, 1)$.

Natural admissible heuristics, especially exact costs of relaxed problems, are usually consistent as well. With $h = 0$, A* graph search is UCS, which is therefore optimal.

A* stores every generated node, so it usually runs out of memory before time. Three variants trade time for space:

- IDA* runs iterative deepening with an f-limit instead of a depth limit. The next limit is the smallest f-value that exceeded the current one. This is inefficient when f is real-valued and nearly every node has a distinct value, since each iteration then adds few nodes.
- RBFS is a recursive depth-first search whose f-limit is the best f-value of any alternative path available from an ancestor of the current node. When the limit is exceeded, the recursion unwinds but remembers the best f-value reachable on the abandoned branch.
- SMA* uses all available memory for the queue. When memory is full, it drops the worst node but records its value in the parent, which limits repeated regeneration (thrashing).

4.2 Local search problems

In many problems the path is irrelevant: the goal configuration itself is the solution. A completed Sudoku grid, for instance, tells us how to reach it. The state space is then a set of complete configurations, and we seek one that satisfies constraints, as in n-queens, or optimizes an objective,

as in the traveling salesperson problem.

Local search (iterative improvement) keeps a single current state, or a few, and tries to improve it by moving to neighboring states. It uses constant space, works online as well as offline, and is largely unavoidable when the state being improved is the agent itself, as in learning. Below, $V(s)$ is the objective to maximize; minimizing a cost h is the same as maximizing $V = -h$.

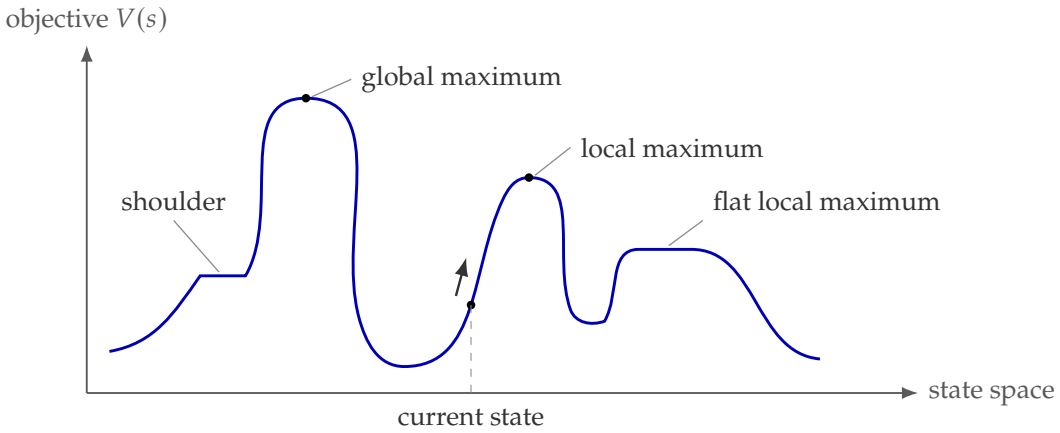


Figure 6: A one-dimensional landscape. Moving uphill from the current state reaches the local maximum, not the global one. Plateaus include flat local maxima and shoulders, from which further progress is possible.

4.3 Hill climbing

Hill climbing (steepest ascent) starts anywhere, repeatedly moves to the best neighbor, and stops when no neighbor is better. It keeps no search tree, only the current state and its value: "like climbing Everest in thick fog with amnesia."

Algorithm 4.1: Hill climbing

Input: Initial state s_0, successors of each state, objective V.
Output: A state with no better successor.

```
s ← s_0
while true do
    s' ← a successor of s maximizing V
    if V(s') ≤ V(s) then
        return s
    s ← s'
```

For n-queens, a state places one queen in each column, an action moves a queen within its column, and h counts attacking pairs: queens sharing a row, column, or diagonal. Hill climbing minimizes h; a goal has $h = 0$.

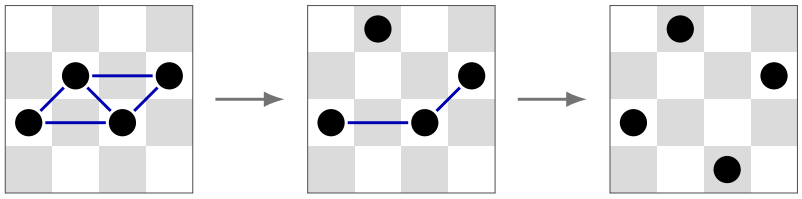


Figure 7: Hill climbing on 4-queens. Lines join attacking pairs. Each arrow moves one queen within its column to a best successor.

Greedy improvement can stop at a LOCAL MAXIMUM, which looks like a global maximum from nearby states, or on a PLATEAU. A plateau is a FLAT LOCAL MAXIMUM if no direction improves, or a SHOULDER if progress resumes after a flat stretch. Hill climbing is therefore incomplete. Common variants are:

- STOCHASTIC HILL CLIMBING: choose randomly among uphill moves. In practice it reaches higher maxima, using more iterations.
- SIDEWAYS MOVES: allow moves of equal value. These escape shoulders but loop forever on flat local maxima, so their number is capped.
- RANDOM-RESTART HILL CLIMBING: rerun from random initial states. If each run succeeds with probability $p > 0$, the number of runs is geometric with mean $1/p$, so a global maximum is found with probability 1.

On 8-queens from random starts, plain hill climbing succeeds with probability $p = 0.14$, averaging 4 moves when it succeeds and 3 when it gets stuck. The expected number of failed runs before success is $(1 - p)/p$, so random restarts need about

$$3 \cdot \frac{1-p}{p} + 4 \approx 22$$

moves in total. Allowing up to 100 sideways moves raises p to 0.94, with averages of 21 and 65 moves, giving $65(1-p)/p + 21 \approx 25$. Each run is far more reliable but longer, and the result depends on a tuning parameter.

4.4 Simulated annealing

SIMULATED ANNEALING combines a random walk with hill climbing, by analogy with slowly cooling a metal into an ordered state. It proposes a random move, always accepts improvements, and accepts a worse move with a probability set by the temperature T. A high temperature permits many bad moves and shakes the state out of local maxima. A schedule lowers T gradually.

Algorithm 4.2: Simulated annealing

Input: Initial state s_0, objective V, schedule $T(t) \geq 0$.
Output: The current state when the temperature reaches zero.

$s \leftarrow s_0$
for $t = 1, 2, \ldots$ **do**
  if $T(t) = 0$ **then**
    return s
  $s' \leftarrow$ a random successor of s
  $\Delta \leftarrow V(s') - V(s)$
  if $\Delta > 0$ **then**
    $s \leftarrow s'$
  else
    $s \leftarrow s'$ with probability $e^{\Delta/T(t)}$

Proposition 4.1 [Boltzmann equilibrium]. *Fix $T > 0$. Suppose the neighbor relation is symmetric, every state has D neighbors, and the proposal is uniform over them. Then*

$$P(x) \propto e^{V(x)/T}$$

is a stationary distribution of simulated annealing.

Proof. Take neighbors x, y with $V(y) \geq V(x)$. The uphill move is always accepted, while the downhill move is accepted with probability $e^{(V(x)-V(y))/T}$:

$$P(x \to y) = \frac{1}{D}, \qquad P(y \to x) = \frac{1}{D} e^{(V(x)-V(y))/T}.$$

With $P(s) = e^{V(s)/T}/Z$, both $P(x)P(x \to y)$ and $P(y)P(y \to x)$ equal $e^{V(x)/T}/(ZD)$. This detailed balance holds on every edge. Rejected proposals leave the state unchanged, so summing over predecessors gives $\sum_y P(y)P(y \to x) = P(x) \sum_y P(x \to y) = P(x)$. □

The ratio $P(y)/P(x) = e^{(V(y)-V(x))/T}$ grows without bound as $T \to 0$, so on a connected state graph the equilibrium concentrates on global maxima. If T decreases slowly enough, the state converges to an optimum with probability approaching 1. The guarantee is weaker than it sounds. Escaping a local maximum may require many consecutive downhill steps, each increasingly unlikely; "slowly enough" can mean exponentially slowly; and random-restart hill climbing has a similar guarantee. Simulated annealing and its relatives are still standard tools for VLSI layout and other configuration problems.

4.5 Local beam search

Local beam search keeps K states, initialized randomly. Each iteration generates all successors of all K states and keeps the best K of the combined list. It stops when any state reaches the optimal value. Stochastic beam search instead samples K successors with a bias toward good ones.

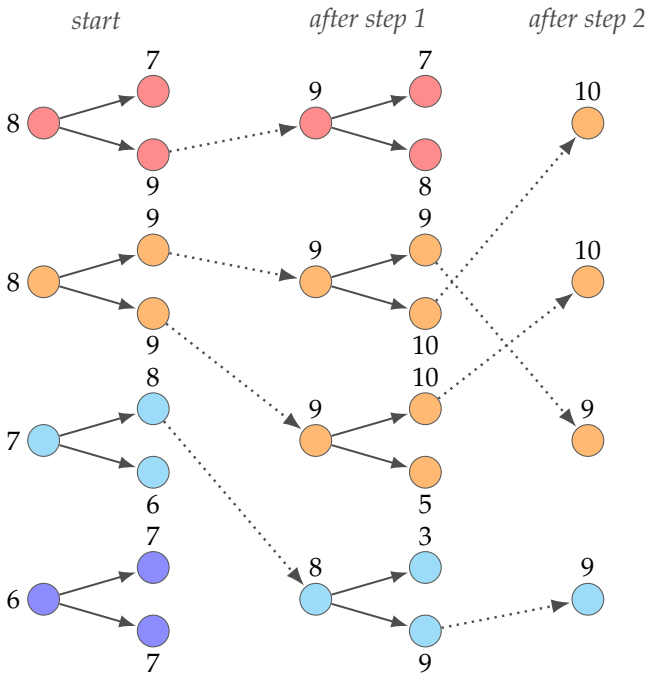


Figure 8: Local beam search with $K = 4$. Numbers are objective values. Colors mark the initial state each state descends from, and dotted arrows show the K best successors kept at each step.

This is not K independent hill climbers. Selection is shared, so productive regions attract the search. In the figure, the lowest initial state is dropped after one step, and after two steps three of the four states descend from one initial state. Beam search can still stall on flat regions; stochastic selection helps, much as in stochastic hill climbing.

4.6 Genetic algorithms

Genetic algorithms are stochastic beam search with a natural selection metaphor. The K states, called the population, are individuals encoded as strings over a finite alphabet. Each step evaluates a nonnegative fitness function, samples parents with probability proportional to fitness, combines pairs by crossover at a random point, and applies occasional random mutation.

For 8-queens, digit i is the row of the queen in column i, from 1 at the bottom. A natural fitness is the number of nonattacking pairs, out of $\binom{8}{2} = 28$, so a solution has fitness 28. In the generation below, the total fitness is 78. Bars mark crossover points, so the first pair yields 327|48552 and 247|52411. Bold digits are mutations.

Population	Fitness	Selection	Parents	Offspring	Mutated
24748552	24	31%	327|52411	32748552	32748**1**52
32752411	23	29%	247|48552	24752411	24752411
24415124	20	26%	32752|411	32752124	32**2**52124
32543213	11	14%	24415|124	24415411	2441541**7**

A mutation moves one queen within its column, a single hill-climbing action. Crossover keeps the left columns of one parent and the right columns of the other. It helps when blocks of the string are good partial solutions, such as nonattacking groups of columns, which evolved separately and can be combined into a high-scoring whole.

Algorithm 4.3: Genetic algorithm with one child per mating

Input: Population P of K strings of length n, fitness $F \geq 0$.
Output: The fittest individual in the final population.

while no individual is fit enough and time remains **do**
  $P' \leftarrow \varnothing$
  for $i = 1$ **to** K **do**
    sample parents x, y from P with probability proportional to F
    choose a crossover point c uniformly from $\{1, \ldots, n\}$
    $z \leftarrow x_1 \cdots x_c \, y_{c+1} \cdots y_n$
    With small probability, mutate a random position of z
    Add z to P'
  $P \leftarrow P'$
return the individual in P maximizing F

The 8-queens example produces two complementary children per pair; this common version keeps one.

4.7 Continuous state spaces

Suppose we place three airports in Romania to minimize the sum of squared distances from each city to its nearest airport. The state is $\mathbf{x} = (x_1, y_1, x_2, y_2, x_3, y_3)$. With city locations (x_c, y_c) and C_a the set of cities nearest airport a,

$$f(\mathbf{x}) = \sum_{a=1}^{3} \sum_{c \in C_a} \big((x_a - x_c)^2 + (y_a - y_c)^2\big).$$

A continuous state or action space can be handled in three ways:

1. *Discretize*: use a grid with increment δ and any discrete local search algorithm.
2. *Random perturbations*: FIRST-CHOICE HILL CLIMBING samples perturbations until one improves the state; simulated annealing also works with random perturbations.
3. *Gradients*: compute $\nabla f(\mathbf{x}) = (\partial f/\partial x_1, \partial f/\partial y_1, \partial f/\partial x_2, \ldots)^\top$ analytically.

Away from ties, small moves do not change the assignments C_a, so

$$\frac{\partial f}{\partial x_1} = \sum_{c \in C_1} 2(x_1 - x_c).$$

An extremum has $\nabla f(\mathbf{x}) = 0$. Here this has a closed form: each airport sits at the centroid of its cities, for example $x_1 = \sum_{c \in C_1} x_c / |C_1|$. For fixed assignments f is a convex quadratic minimized by these centroids. Without ties, the assignments stay fixed under small moves, so this is a local minimum of f. Other assignments can give other local minima, so it need not be global.

When $\nabla f = 0$ has no closed form, GRADIENT DESCENT repeats

$$\mathbf{x} \leftarrow \mathbf{x} - \alpha \nabla f(\mathbf{x})$$

with step size $\alpha > 0$. This is hill climbing that minimizes a cost. Many gradient-based methods refine this idea, and many machine learning algorithms are local searches of this kind.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 4 (Spring 2024); Nikhil Sharma, Note 4.